

IEP 2026 — Investment Fund System

How it works and how to modify it
A guide written for a Java/C#-minded reader

Check	Result
Official grader (iep_grader/), simple mode — combination 2	180.00 / 180.00 (100%)
Official grader (iep_grader/), voting mode — combination 1	179.00 / 179.00 (100%)
Own end-to-end script (verify/verify.py), both modes	all checks pass

Verified 2 July 2026, docker-compose deployment, fresh databases.

Contents

- 1 The system in one page
- 2 Reading the Python when you think in Java
- 3 Service walkthroughs (authentication, employee, director)
- 4 The voting mode and the smart contract
- 5 Configuration — the one place everything is changed
- 6 Running it and grading it
- 7 Modification cookbook — common changes, step by step
- 8 Troubleshooting on this machine

1 The system in one page

Three small HTTP services (in Java terms: three separate Spring Boot applications, each in its own container), three databases, and an optional local Ethereum chain:

authentication/	:5000	/register /login /delete	-> MySQL (user accounts)
employee/	:5001	/search /create_buy_order /create_sell_order	-> MongoDB + Redis
director/	:5002	/pending_orders /decision /report	-> MongoDB + Redis (+ ganache)
ganache	:8545	throw-away Ethereum chain, voting mode only	

Each service folder is self-contained: `app.py` (all endpoints — think one `@RestController` class), `configuration.py` (application.properties), `requirements.txt` (pom.xml) and a `Dockerfile`. There is no shared code between services, so you can edit and rebuild any one of them in isolation.

The life of an order

1. An **employee** calls `/create_buy_order` or `/create_sell_order`. The order is *not* written to the main database yet — it is stored in **Redis** as a JSON string under the key `order:<uuid>`. Redis here plays the role of a small shared in-memory map of pending work.
2. The **director** lists those keys with `GET /pending_orders` and decides each one with `POST /decision`. In **simple** mode the body is `{uuid, approved}`; in **voting** mode it is `{uuid, voters}` and the decision is made later by a majority vote on a smart contract.
3. On approval the order is applied to **MongoDB** (a BUY inserts an asset document with `buying_date = now`; a SELL sets `selling_price/selling_date` on an existing document) and the Redis key is deleted. On rejection the key is just deleted.
4. `/search` and `/report` only ever read MongoDB. All filtering and aggregation is pushed into the database as one query — the Python code never loops over assets in memory.

Who may call what

Every endpoint requires a JWT issued by the authentication service (HTTP header `Authorization: Bearer <token>`). The token carries a role claim — `employee` endpoints and `director` endpoints check it. A missing header *and* a wrong-role token both get the same reply, `401 {"msg": "Missing Authorization Header"}` (deliberate: the spec only defines the missing-header reply, and answering identically leaks nothing; the grader accepts it — see section 3).

2 Reading the Python when you think in Java

You don't need to like Python to modify this project — the code base is about 700 lines total and uses a small, repetitive set of idioms. This table is the whole dictionary you need:

You see in the code	Think (Java/C# equivalent)
<code>@application.route("/search", methods=["POST"])</code>	<code>@PostMapping("/search")</code> on a controller method
<code>@roles_required("employee")</code>	<code>@PreAuthorize("hasRole('EMPLOYEE')")</code> — a custom annotation defined at the top of each app.py
<code>body = request.get_json(silent=True) or {}</code>	<code>@RequestBody Map<String,Object> body;</code> 'or {}' = default to empty map if the body isn't JSON
<code>body.get("name")</code>	<code>map.get("name")</code> — returns None (null) when absent, never throws
<code>return jsonify(message="..."), 400</code>	<code>return ResponseEntity.status(400).body(Map.of("message", ...))</code>
<code>def error(message): ...</code>	a private static helper method; functions can live outside classes
<code>f"Field {name} is missing."</code>	<code>String.format / interpolation: \$"Field {name} is missing."</code>
<code>isinstance(value, bool)</code>	<code>value instanceof Boolean</code> — needed because Python booleans are untyped maps
<code>for key in orders_store.scan_iter("order:*")</code>	<code>for (String key : redis.scan("order:*"))</code> — a lazy iterator
<code>{"\$gt": value}</code> etc. (dict literals)	<code>Map.of("\$gt", value)</code> — MongoDB queries are literally JSON maps
<code>with open(...) as f:</code>	<code>try-with-resources</code>
<code>ClassName()</code> — no 'new'	<code>new ClassName()</code>

Two things that genuinely differ from Java and that the code exploits:

- **Indentation is the braces.** A block ends where the indentation ends. When editing, keep 4-space indentation exactly — mixing tabs and spaces is a syntax error.
- **bool is a subtype of int** (so `isinstance(True, int)` is true!). That is why `is_valid_price` in `employee/app.py` explicitly excludes bool — otherwise `"buying_price": true` would count as a valid price of 1.

There is no compiler to catch mistakes; a typo in a rarely-executed branch only explodes at runtime. After any edit, rebuild and rerun the grader (section 6) — it exercises every branch, which is your type check here.

3 Service walkthroughs

3.1 authentication/app.py

Users live in MySQL, accessed through SQLAlchemy (Hibernate/JPA equivalent; the User class with class-level Column attributes is an @Entity). Passwords are stored as werkzeug hashes.

- **Startup auto-init:** the service retry-loops until MySQL answers, then runs `create_all()` (schema from the entity classes) and inserts the seeded director `onlymoney@gmail.com` / `evenmoremoney` (Scrooge McDuck) if absent. Idempotent, works the same under compose and Kubernetes.
- **/register:** validation runs in a fixed order the grader checks literally: missing forename → surname → email → password (a field is missing when absent or an empty string — a lone space " " counts as present!) → email regex (TLD must be ≥ 2 letters, so `john@gmail.a` is invalid) → password length ≥ 8 → duplicate email. Success: 200, empty body, role is always employee.
- **/login:** missing email → missing password → email format → credentials. Returns `{"accessToken": ...}`; the JWT (flask-jwt-extended) expires in exactly 1 hour and carries claims `forename`, `surname`, `role`, `sub=email`. The grader decodes it with the shared secret and checks each claim, so don't rename claims.
- **/delete:** token required; deletes the token's user. A token whose user is already gone gets 400 `{"message": "Unknown user."}`.

3.2 employee/app.py

- **/search** builds one MongoDB `find()` query from the optional body fields (`build_search_query`, ~line 99): `name` → escaped `$regex` substring; `category` → array membership; `buying_date` → `$gt` (bought after); `selling_date` → `$lt` (sold before — unsold documents lack the field, so they drop out automatically); each `info_filter {field, operator, value}` → `{"info.<path>": {"$op": value}}`. Dot-paths like `geo.country` work because MongoDB natively resolves them into nested documents. Operators arrive as `eq/gt/lt/...` and get a `$` prefixed.
- **/create_buy_order:** missing name → categories → `buying_price` → `info` → "Categories list is empty." → "Invalid buying price." (must be a number > 0, bool excluded). Stores `{order_type: BUY, name, categories, buying_price, info}` in Redis.
- **/create_sell_order:** missing id → missing `selling_price` → "Invalid id." (must be a well-formed Mongo ObjectId AND exist in the `assets` collection) → "Invalid selling price.". Stores `{order_type: SELL, id, selling_price}`.

Inspecting pending orders during a demo: `docker exec -it iep-redis-1 redis-cli`, then `KEYS order:*` and `GET <key>` — the values are plain JSON.

3.3 director/app.py

- **/pending_orders:** scans Redis for `order:*` keys, parses each JSON value, adds the uuid (the key without the prefix) and returns `{"orders": [...]}`.
- **/decision** dispatches on the `DECISION_MODE` environment variable (a strategy switch — `decision_simple` vs `decision_voting`). **Check order matters and the grader tests it:** the uuid is fully validated first — "Field uuid is missing." → "Invalid uuid." (bad format or not a pending order) — and only then the second field: `approved` ("Field approved is missing." → "Invalid decision." if not boolean) in simple mode, or `voters` ("Field voters is missing." for absent/empty → "Invalid voter address." → "Even number of voters.") in voting mode. This exact order was the one thing the official grader flagged (2 tests) and was fixed on 2 July 2026.
- Both modes funnel into the same two functions: `apply_order` (the Mongo write; `buying/selling date` = moment of approval) and `conclude_order` (apply if accepted, then delete the Redis key). Approval

semantics exist exactly once — change them there and both modes follow.

- **/report** is a single MongoDB aggregation pipeline (SQL in comments): \$unwind categories (one row per asset-category pair) → \$group per category (spent = sum of all buying_price, earned = sum of selling_price with \$ifNull → 0 for unsold) → \$sort earned desc, spent asc, category asc. An asset in two categories counts fully toward both — that is what the spec and grader expect.

4 The voting mode and the smart contract

Only active when `DECISION_MODE=voting`. The web3/contract code lives in `director/voting.py` and is imported only in that mode — simple mode runs without ganache or the web3 library entirely.

- **Contract** (`director/contracts/Voting.sol`): constructor takes the odd-sized voter list; `vote(bool)` requires the sender to be a listed voter (`revert "Invalid address."`), voting not finished (`"Voting ended."`), one vote per address; the first side reaching $n/2+1$ finishes the vote. Solidity reads like C/Java, so this is the file you'll find most familiar.
- **Deployment**: `/decision` deploys one fresh contract per order, paid by ganache account 0, and returns unsigned `approve_transaction` / `reject_transaction` objects that voters sign and send themselves. The mapping `vote:<uuid> → contract address` is kept in Redis, so a director restart loses nothing.
- **Watcher**: a daemon thread polls each active contract every `VOTE_POLL_INTERVAL` (default 2 s); when a contract reports finished it calls `conclude_order` — the same code path as a simple-mode approval.
- **Compile step**: the contract is compiled at image build time (`compile_contract.py` runs a static `solc 0.8.19` binary downloaded in the Dockerfile). The running container needs no compiler and no internet. If you edit `Voting.sol` you must rebuild the director image (or rerun `compile_contract.py`) — the `.json` artifact is what gets deployed, not the `.sol`.

Nonce note: the returned unsigned transactions carry nonce 0, correct for fresh ganache voter accounts; a voter that has already transacted replaces the nonce before signing (the grader does exactly that).

5 Configuration — the one place everything is changed

Every setting is an environment variable with a localhost-friendly default, collected in one `configuration.py` per service (its `application.properties`). `docker-compose.yml` sets them for compose; `kubernetes.yaml` sets the same names through a `ConfigMap` (plain values) and a `Secret` (passwords). You never edit service code to reconfigure — you edit compose/k8s or set the variable.

Variable	Used by	Meaning / default
<code>DATABASE_HOST/PORT/USERNAME/PASSWORD/DATABASE</code>	authentication	MySQL connection (localhost:3306, root/root, db 'authentication')
<code>DIRECTOR_FORENAME/SURNAME/EMAIL/PASSWORD</code>	authentication	seeded director account (Scrooge McDuck / onlymoney@gmail.com / evenmoremoney)
<code>MONGO_URI, MONGO_DATABASE</code>	employee, director	MongoDB (mongodb://localhost:27017, db 'fund')
<code>REDIS_HOST, REDIS_PORT</code>	employee, director	Redis (localhost:6379)
<code>JWT_SECRET_KEY</code>	all three	must be identical everywhere; default <code>JWT_SECRET_DEV_KEY</code>
<code>DECISION_MODE</code>	director	'simple' (default) or 'voting'; <code>BLOCKCHAIN_ENABLED=true</code> is accepted as an alias
<code>GANACHE_URL</code>	director	voting mode only; <code>http://localhost:8545</code>
<code>VOTE_POLL_INTERVAL</code>	director	watcher poll period in seconds, default 2
<code>SERVICE_PORT</code>	each	listen port: 5000 / 5001 / 5002

Hard-coded (deliberately, shared constants): the Redis key prefixes `order:` and `vote:`, and the 1-hour token lifetime (`JWT_ACCESS_TOKEN_EXPIRES` in `authentication/configuration.py` — the spec requires exactly 1 h, so it is not an env var).

6 Running it and grading it

docker-compose (development)

```
docker compose up --build -d          # everything, simple mode
docker compose down -v                # stop AND wipe all data
```

```
# voting mode (fresh data + DECISION_MODE=voting):
docker compose down -v
$env:DECISION_MODE = "voting"; docker compose up -d
# back to simple mode: close the shell or $env:DECISION_MODE = $null
```

Host ports: auth 5000, employee 5001, director 5002, ganache 8545, MySQL 3307, Mongo 27018 (3306/27017 are taken by other software on this machine), Redis 6379.

Kubernetes (the assignment's target)

```
docker compose build                  # builds iep/authentication|employee|director
kubectl apply -f kubernetes.yaml
# NodePorts: auth 30000, employee 30001 (3 replicas), director 30002, ganache 30545
# switch decision mode:
kubectl patch configmap iep-config --type merge -p '{"data":{"DECISION_MODE":"voting"}}'
kubectl rollout restart deployment director
```

The official grader (iep_grader/)

The grader is stateful and assumes empty databases. It registers users, creates named assets (AssetAlpha, AssetBeta, ...) and asserts exact report totals; leftovers from a previous run mean bogus failures. Always reset first: `docker compose down -v; docker compose up -d`.

One-time setup of its virtual environment (a venv is a project-local library directory — closest Java analogy: a private local Maven repository):

```
python -m venv iep_grader\.venv
iep_grader\.venv\Scripts\pip install -r iep_grader\requirements-pytest.txt
iep_grader\.venv\Scripts\pip install "setuptools<81" # Python 3.13 fix, see below
```

Combination 2 — authentication, no blockchain (simple mode):

```
iep_grader\.venv\Scripts\pytest.exe iep_grader -q --type all `
  --authentication-url http://127.0.0.1:5000 --jwt-secret JWT_SECRET_DEV_KEY `
  --roles-field role --employee-role employee --director-role director `
  --with-authentication --employee-url http://127.0.0.1:5001 `
  --director-url http://127.0.0.1:5002 --wait-for-services
# expected: TOTAL: 180.00/180.00 (100.00%)
```

Combination 1 — authentication + blockchain (voting mode; stack must be started with `DECISION_MODE=voting`): same command plus

```
--with-blockchain --provider-url http://127.0.0.1:8545
# expected: TOTAL: 179.00/179.00 (100.00%)
```

Useful extras: `--html=report.html` `--self-contained-html` for a readable report, `--type level1` to run only up to a level while debugging. `grade_report.json` is always written. The old tests/Tests folder is last year's unrelated project — ignore it. `verify/verify.py` remains as a second, independent check (venv in `verify/.venv`, see `SUMMARY.md`).

7 Modification cookbook

Golden rule for every change: `edit → docker compose down -v; docker compose up --build -d` → rerun the grader. The build is fast (layers are cached; only the changed service is rebuilt).

7.1 Change an error message or a status code

All error replies go through a one-line helper in each `app.py`: `def error(message): return jsonify(message=message), 400`. Individual messages are the literal strings at each validation site (grep the message text). Change the string, rebuild that one service. Beware: the grader matches messages exactly, byte for byte, including the trailing period.

7.2 Add a field to registration (example: phone number)

Everything happens in `authentication/app.py`:

1. Add a column to the User class: `phone = database.Column(database.String(32), nullable=False)` — like adding an `@Column` field to the entity. 2. Add "phone" to the required-fields list in `/register` (position in that list = position in the validation order). 3. Pass it when constructing the User. 4. Optionally add it to the token claims in `/login`. 5. Because the schema changed, wipe MySQL once: `docker compose down -v`.

7.3 Add a whole new endpoint

```
@application.route("/my_endpoint", methods=["POST"])
@roles_required("director")          # or "employee", or drop the line for public
def my_endpoint():
    body = request.get_json(silent=True) or {}
    # ... validate like the neighbours do, reusing find_missing_field/error ...
    return jsonify(result="ok"), 200
```

Paste next to the existing routes in the right service's `app.py`; no `registration/DI/web.xml` step exists — the decorator is the registration.

7.4 Change validation order or rules

The order of checks is simply the order of if-statements (and the order of names in each `find_missing_field([...])` list). The grader tests specific orders with carefully crafted partial bodies — if you reorder anything, rerun it. The `/decision` order (uuid fully validated before approved/voters) is exactly what the grader demands; don't 'clean it up' back to checking all missing fields first.

7.5 Ports, passwords, hostnames, mode, token lifetime

Want to change	Edit
Host ports, DB names, hostnames	<code>docker-compose.yml</code> (ports:/environment:) and the <code>ConfigMap</code> in <code>kubernetes.yml</code>
Passwords, JWT secret	environment in <code>docker-compose.yml</code> and the <code>Secret</code> in <code>kubernetes.yml</code> (and pass the new secret to the grader via <code>--jwt-secret</code>)
simple vs voting	<code>DECISION_MODE</code> env var (compose shell var / k8s <code>ConfigMap</code> patch)
Token lifetime	<code>JWT_ACCESS_TOKEN_EXPIRES</code> in <code>authentication/configuration.py</code> (spec says 1 h)
Database engine (e.g. Postgres)	<code>SQLALCHEMY_DATABASE_URI</code> in <code>authentication/configuration.py</code> + driver in <code>requirements.txt</code> ; all access is via the ORM so no query changes

Want to change	Edit
Voting rules (majority, quorum...)	director/contracts/Voting.sol, then rebuild the director image (auto-recompiles)
Watcher poll frequency	VOTE_POLL_INTERVAL env var
Wrong-role reply 401 → 403	roles_required in each app.py — a one-line change, but the grader expects 401, so don't for grading

7.6 Change what /report computes

The whole report is the pipeline list in director/app.py (report()). Each stage is a map: add e.g. a count per category by putting "count": {"\$sum": 1} inside the \$group stage, and expose it by adding "count": 1 to the \$project stage. MongoDB aggregation docs map 1:1 onto these dict literals.

8 Troubleshooting on this machine

Symptom	Cause / fix
docker build fails with 'exec /bin/sh: exec format error'	A wrong-architecture base image ended up in the BuildKit cache (happened 2 July 2026). Fix: docker builder prune -af; docker pull python:3.10-slim; rebuild.
Grader fails with ModuleNotFoundError: pkg_resources	web3 6.5.0 needs pkg_resources, removed in setuptools >= 81 (and Python 3.13 venvs ship no setuptools at all). Fix: iep_grader\.venv\Scripts\pip install "setuptools<81".
Grader fails on 'search: empty result' or wrong report totals	Stale data — the grader needs empty databases. docker compose down -v; docker compose up -d and rerun.
Ports 3306 / 27017 already in use	Expected on this machine (native mysqld, another app's Mongo). Compose intentionally maps MySQL to 3307 and Mongo to 27018 on the host.
Docker Desktop won't start after a force-kill	Stale socket files. Rename %LOCALAPPDATA%\Docker\run (and %APPDATA%\Docker\run if present) before starting Docker Desktop again.
Voting decision never applies	Contract needs a strict majority: with 3 voters, 2 must vote the same way. Also check the ganache container is up and GANACHE_URL is reachable from the director.
401 on everything	Token expired (1 h) or JWT_SECRET_KEY differs between services. All three services must share the secret.

Companion files: SUMMARY.md in the project root holds the same knowledge in markdown form (kept up to date), IEP_Projekat_2026.pdf is the assignment specification, iep_grader/README.md documents every single grader test case with its exact request and expected response — when a test fails, that file tells you precisely what was expected.